



COMSATS UNIVERSITY ISLAMABAD

Vehari Campus

Department of Computer Science

PROJECT PROPOSAL

AI News Summarizer Agent

Submitted To	Dr.Tahir Rasheed
Submitted By	Muhammad Umer Asad Muhammad Sami Ullah
Registration No	FA24-BSE-084 FA24-BSE-080
Course	Artificial Intelligence Lab
Section	FA24-BSE-4B

1. Introduction

1.1 What is Text Summarization?

Text summarization is the process of shortening a text while preserving its main ideas. It is an important task in Natural Language Processing (NLP) that helps readers understand content quickly.

There are two main types of summarization:

Extractive Summarization

Extractive summarization selects important sentences from the original text to create a summary. It is simple, reliable, and transparent, which is why it is used in this project.

Abstractive Summarization

Abstractive summarization generates new sentences to convey the meaning of the original text. Although more advanced, it is harder to build and explain, so this project does not use it.

1.2 Why is News Summarization Important?

With the rapid growth of online news, reading every article in detail is difficult. News summarization helps by presenting the key points in a short and readable form.

Automatic news summarizers are especially useful for:

- Students and researchers who need to stay informed quickly
- Busy professionals who want concise news highlights
- News aggregator applications that process large volumes of content
- Systems that monitor media for key events or topics

1.3 Project Overview

This project presents the AI News Summarizer Agent, a rule-based extractive summarization system developed in Python. It identifies key sentences from English news articles and generates clear summaries through a simple Tkinter-based interface.

2. Problem Statement

The modern reader faces significant information overload. Thousands of news articles are published every hour, making it impossible to read every article in full. At the same time, most existing summarization solutions either:

- Rely on expensive, complex machine learning models inaccessible to beginners, or
- Function as black-box systems that cannot be easily inspected, understood, or explained.

This project addresses the gap by building a system that:

- Accepts any English news article as plain text input
- Automatically identifies and ranks the most important sentences
- Generates a meaningful short summary without human intervention
- Uses fully explainable, rule-based logic — no opaque ML models
- Is simple enough for a university student to build, understand, and present

3. Objectives

- Develop a fully rule-based extractive summarization system without machine learning.
- Apply core NLP techniques: tokenization, stop-word removal, and word-frequency analysis.
- Implement a multi-factor sentence scoring system considering frequency, position, title keywords, and named entities.
- Reduce summary redundancy using Jaccard similarity-based filtering.
- Automatically detect the news category using weighted keyword matching across nine domains.
- Build a user-friendly graphical interface using Python's Tkinter library.
- Produce a system that is fully explainable, configurable, and suitable for academic demonstration.

4. Project Scope

4.1 In Scope

- Automatic summarization of English-language news articles of any length
- Extractive sentence ranking based on multiple rule-based scoring factors
- News-category detection across nine domains: Technology, Politics, Sports, Business, Health, Science, World, Crime, and Education
- Keyword-frequency report and sentence-ranking analytics
- Export of full analysis report to a plain-text file
- Tabbed GUI panels for summary, keywords, sentence scores, and statistics

4.2 Out of Scope

- Abstractive summarization or new sentence generation
- Summarization of non-English articles
- Real-time news fetching from online sources
- Machine learning, deep learning, or transformer-based models of any kind

5. Technologies Used

Technology / Library	Role in Project
Python 3	Primary programming language — readable, beginner-friendly, rich NLP ecosystem
NLTK	Sentence tokenization (Punkt) and built-in English stop-word list (~180 words)
re (regex)	Text preprocessing: remove URLs, smart quotes, dashes, special characters
collections.Counter	Efficient word-frequency counting and normalization
Tkinter	Built-in GUI library — tabbed output panels, keyword bar charts, save-to-file
string module	Punctuation filtering during stop-word removal phase

6. System Architecture

6.1 File Structure

The project consists of two Python files with clean separation of concerns:

`summarizer.py` — Core rule-based NLP engine (all AI logic)

`gui.py` — Graphical user interface (Tkinter frontend)

6.2 Nine-Stage Processing Pipeline

When the user clicks Summarize, the following stages execute in sequence:

- Text Preprocessing — remove URLs, normalize quotes/dashes, collapse whitespace
- Sentence Tokenization — NLTK Punkt tokenizer; sentences under 6 words discarded
- Stop-Word Removal — NLTK list + domain-specific filler words removed
- Word Frequency Calculation — Counter + normalization to 0.0–1.0 scale
- Base Sentence Scoring — average normalized frequency of content words per sentence
- Heuristic Boosts — position bonus, title-keyword match bonus, named-entity bonus
- Length Penalty — sentences over 45 words receive a proportional penalty
- Redundancy Reduction — Jaccard similarity filter (threshold 0.45) removes near-duplicates
- Summary Assembly — selected sentences re-ordered to original article position for natural flow

7. Methodology

7.1 Scoring Formula

Each sentence's final importance score is calculated as:

```
Score = BaseFrequency + PositionBoost + TitleBoost + EntityBoost - LengthPenalty
```

7.2 Heuristic Boost Values

- Position Boost: +0.30 (sentence 1), +0.20 (sentence 2), +0.10 (sentences 3–5)
- Title Keyword Boost: +0.15 per matching headline keyword
- Named Entity Boost: +0.12 per detected entity (person names, countries, acronyms, organisations) — capped at 3 entities
- Length Penalty: $0.08 \times (\text{excess_words} \div 45)$ for sentences exceeding 45 words

7.3 Redundancy Reduction

After ranking, the system applies Jaccard similarity filtering: $J(A,B) = |A \cap B| / |A \cup B|$. Any candidate sentence sharing more than 45% content-word overlap with an already-selected sentence is skipped, ensuring diverse topic coverage across the summary.

7.4 Category Detection

A three-tier weighted keyword system classifies each article: high-signal phrases score 3 points, medium keywords score 2, and low-signal words score 1. Nine categories are supported: Technology, Politics, Sports, Business, Health, Science, World, Crime, and Education. Priority-based tie-breaking resolves ambiguous articles.

8. Expected Results & Sample Output

For a 273-word Science article (NASA Mars discovery), the system produces:

Metric	Value
Original Word Count	273 words
Summary Word Count	73 words
Compression Ratio	26.7%
Detected Category	Science (confidence: 15)

9. Advantages of Rule-Based AI

Full Explainability

Every decision made by the system can be fully traced and explained. The score of any sentence can be broken down into its exact components: base frequency score, position boost, title boost, entity boost, and length penalty. There are no hidden weights, no neural network layers, and no black-box computations — making it ideal for academic study and presentation.

No Training Data Required

Machine learning models require large labelled datasets to train, which are expensive and time-consuming to prepare. This rule-based system requires no training data whatsoever and works on any article immediately without any prior learning phase.

Lightweight and Fast

The system runs entirely on a standard laptop CPU with minimal memory usage. There is no GPU requirement, no large model file to load, and no API call to an external server. Summarization of a typical 500-word article completes in under one second.

Fully Configurable

All scoring weights are defined as named constants at the top of the code. A developer can change the position boost, entity boost, similarity threshold, or length penalty simply by editing a number — no architectural changes needed.

10. Limitations

- No deep contextual understanding — scoring is based on frequency and pattern matching, not semantic meaning.
- Weaker than transformer-based models (BERT, GPT) which understand language semantics.
- English-only — stop words, regex patterns, and category keywords support English text only.
- Dependent on hand-crafted rules — adding a new language or entity type requires manual coding.
- Cannot generate new sentences — output is limited to sentences already present in the article.

11. Future Improvements

Enhancement	Description
TF-IDF Scoring	Replace simple frequency with TF-IDF to reward words unique to the article — still fully rule-based.
Real-Time News	Integrate with NewsAPI or RSS feeds to auto-fetch and summarize latest articles.
Multilingual Support	Add stop-word lists and patterns for Urdu, Arabic, French using NLTK's built-in language resources.
Hybrid AI Approach	Combine rule-based summarization with lightweight classifier (Naive Bayes) for improved category accuracy.
Export Options	Allow direct PDF export, clipboard copy, and email/share functionality from within the GUI.
Sentence Compression	Rule-based removal of relative clauses and parentheticals to shorten extracted sentences.

12. Conclusion

This project successfully demonstrates the application of rule-based Artificial Intelligence and Natural Language Processing to the practical problem of automatic news summarization. Developed entirely in Python using open-source libraries, the system requires no machine learning, no training data, and no external services.

The nine-stage pipeline — from text preprocessing to Jaccard similarity-based redundancy reduction — produces concise, readable, and informative summaries. The multi-factor scoring system combining word frequency, position boosts, title keyword matching, and named-entity detection provides a more accurate sentence ranking than simple frequency-based methods alone.

The project meets all stated objectives and delivers a working, user-friendly application that demonstrates key NLP concepts — tokenization, frequency analysis, heuristic scoring, and similarity measurement — through transparent, beginner-accessible, and fully explainable code. It is suitable for academic demonstration and serves as a strong learning reference for students beginning their journey in Artificial Intelligence and Natural Language Processing.